

API Architecture — FastAPI, Dependency Injection & SSE Streaming

Tài liệu giải thích kiến trúc API backend, bao gồm cách tổ chức module, Dependency Injection pattern, và cơ chế Server-Sent Events cho streaming response.

Tham chiếu mã nguồn:

- `api/main.py` — FastAPI entry point
 - `src/chat/router.py` — Chat endpoints
 - `src/chat/dependencies.py` — Lazy-init DentalChatbot
 - `src/auth/router.py` — Auth endpoints
-

1. Kiến trúc Module

```
api/  
└─ main.py          ← Entry point, CORS, include routers  
  
src/  
├─ chat/  
│   ├── router.py    ← Chat API endpoints  
│   ├── schemas.py   ← Pydantic request/response models  
│   └─ dependencies.py ← Lazy-init chatbot (DI provider)  
├─ auth/  
│   ├── router.py    ← Auth API endpoints  
│   ├── schemas.py   ← Pydantic models  
│   └─ utils.py      ← JWT, bcrypt, get_current_user  
├─ agent/  
│   └─ chatbot.py    ← RAG pipeline logic  
├─ retriever/  
│   ├── search.py    ← Hybrid search  
│   ├── ingest.py    ← FAISS index builder  
│   └─ engines.py    ← Embedding engines  
├─ database/  
│   ├── database.py  ← SQLAlchemy engine & session  
│   └─ models.py     ← ORM models  
├─ lib/  
│   └─ constants.py  ← All prompts & AI config  
└─ config.py         ← Centralized .env config
```

Nguyên tắc tổ chức

- **Domain-driven:** Mỗi domain (chat, auth) là 1 package riêng với router + schemas + dependencies
 - **Separation of Concerns:** `main.py` chỉ setup app + CORS + include routers, không chứa business logic
 - **Config tập trung:** Mọi biến môi trường qua `config.py`, mọi prompt qua `constants.py`
-

2. FastAPI Entry Point

```
# api/main.py (~43 dòng)
@asynccontextmanager
async def lifespan(app: FastAPI):
    # Pre-warm: load FAISS index + sbert model + BM25 corpus ngay khi server start
    print("[STARTUP] Đang khởi tạo DentalChatbot + load Embedding model...")
    get_chatbot()
    print("[STARTUP] Sẵn sàng nhận request.")
    yield

app = FastAPI(title="Dental AI API", version="1.0.0", lifespan=lifespan)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(auth_router.router)  # /api/auth/*
app.include_router(chat_router.router)  # /api/chat/*
```

`main.py` chỉ làm 4 việc:

1. Tạo database tables (`models.Base.metadata.create_all`) — idempotent, đảm bảo schema có sẵn
2. Dùng `lifespan` để **pre-warm** `DentalChatbot` ngay khi server start (load model + BM25 corpus) → request đầu tiên không bị cold-start
3. Cấu hình CORS middleware (mở cho mọi origin trong dev; **nên siết lại** cho production)
4. Include 2 routers: `/api/auth/*` và `/api/chat/*`

3. Dependency Injection — Lazy Initialization

Vấn đề

`DentalChatbot` khi khởi tạo sẽ:

- Load FAISS index (~50MB)
- Load sentence-transformers model (~420MB)
- Khởi tạo BM25 corpus (tokenize 762 tài liệu)

Nếu khởi tạo ngay khi server start → chậm startup. Nếu khởi tạo mỗi request → lãng phí tài nguyên.

Giải pháp: Singleton + Lifespan Pre-warm

```
# src/chat/dependencies.py
_chatbot: DentalChatbot | None = None

def get_chatbot() -> DentalChatbot:
    """Singleton – chỉ khởi tạo DentalChatbot 1 lần duy nhất."""
    global _chatbot
    if _chatbot is None:
        t0 = time.perf_counter()
        _chatbot = DentalChatbot()
        print(f"[STARTUP] DentalChatbot khởi tạo xong trong {time.perf_counter() - t0:.2f}s")
    return _chatbot
```

- **Singleton:** Các request sau dùng chung instance, không tạo mới
- **DI-compatible:** Inject qua `Depends(get_chatbot)` trong FastAPI
- **Pre-warm:** `api/main.py` gọi `get_chatbot()` ngay trong `lifespan` trước khi nhận request đầu tiên → tránh cold-start (~5–8 s) cho người dùng đầu tiên

Sử dụng trong Router

```
@router.post("")
async def chat(
    request: ChatRequest,
    db: Session = Depends(get_db),
    current_user = Depends(get_current_user),
    chatbot: DentalChatbot = Depends(get_chatbot),
):
    ...
```

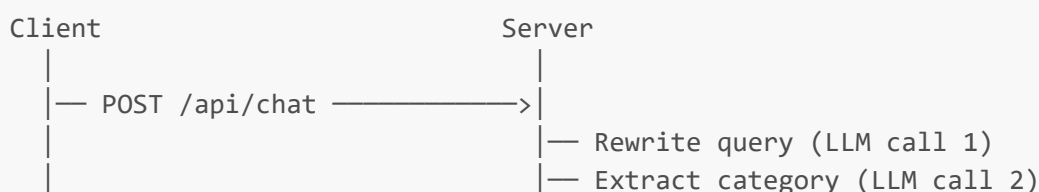
FastAPI tự động gọi `get_chatbot()`, `get_db()`, `get_current_user()` trước khi handler chạy.

4. Server-Sent Events (SSE) — Streaming Response

Tại sao cần SSE?

LLM sinh câu trả lời **từng token** (mỗi token ~0.05s). Nếu đợi toàn bộ xong mới trả → người dùng chờ 5-10 giây nhìn màn hình trắng. SSE cho phép **stream từng phần** → người dùng thấy chữ hiện dần như chatbot đang "gõ".

Cơ chế hoạt động



```

|                                     |— Hybrid search
|                                     |— Start answer stream (LLM call 4)
|
|<— data: {"token": "Đề"} ———|
|<— data: {"token": " chăm"} —|
|<— data: {"token": " sóc"} —|
|      ...
|<— data: {"token": "..."} ———|
|<— data: {"done": true, ———|
|      "sources": [...],
|      "rewritten_query": "..."}|
|

```

Triển khai

```

async def stream():
    full_answer = ""
    sources = []

    for item in chatbot.answer_stream(user_question, chat_history):
        if isinstance(item, str):
            full_answer += item
            yield f"data: {json.dumps({'token': item})}\n\n"
            await asyncio.sleep(0.01)
        elif isinstance(item, dict):
            sources = item.get("sources", [])

    _save_message(db, session_id, user_id, "assistant", full_answer, sources)
    yield f"data: {json.dumps({'done': True, 'sources': sources})}\n\n"

    return StreamingResponse(stream(), media_type="text/event-stream")

```

Đặc điểm:

- `media_type="text/event-stream"` — chuẩn SSE
- Format: `data: {JSON}\n\n` — mỗi event cách nhau bằng 2 newline
- `asyncio.sleep(0.01)` — nhường event loop, tránh block
- Message cuối: `{"done": true, "sources": [...]}` — báo hiệu kết thúc stream
- Database save xảy ra **sau khi stream xong** — không block streaming

Xử lý lỗi

```

except Exception as e:
    yield f"data: {json.dumps({'error': 'Lỗi trong quá trình tạo câu trả lời'})}\n\n"

```

Nếu LLM lỗi giữa chừng, client nhận event `error` và hiển thị thông báo thay vì treo.

5. Pydantic Schemas — Validation

```
# src/chat/schemas.py
class ChatMessage(BaseModel):
    role: str
    content: str

class ChatRequest(BaseModel):
    user_question: str
    session_id: str
    chat_history: list[ChatMessage] = []
```

Pydantic tự động:

- Validate kiểu dữ liệu (string, list, ...)
- Reject request thiếu field bắt buộc
- Cung cấp default value cho field optional
- Generate OpenAPI schema cho Swagger docs

6. Danh sách API Endpoints

Chat (/api/chat)

Method	Endpoint	Mô tả	Response
POST	/	Gửi câu hỏi, nhận SSE stream	text/event-stream
GET	/sessions	Danh sách phiên hội thoại	JSON array
GET	/sessions/{id}/messages	Lịch sử chat của 1 phiên	JSON array
DELETE	/sessions/{id}	Xóa 1 phiên	JSON message
DELETE	/sessions	Xóa toàn bộ lịch sử	JSON message

Tất cả endpoints đều yêu cầu JWT token (Depends(get_current_user)).